SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1504

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**
Question Count: 5

Code of Conduct

I **Edrin J S** (Name / Reg No) **192524041** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Edrin J S*
Date: 29 July 2026 **Batch:** 2025

Criteria	Problem Interpretation (2)	Analytical Reasoning (2.5)	Method Selection (2)	Solution Quality (2)	Presentation of Analysis (1.5)	TOTAL (10)
Weightage	20%	25%	20%	20%	15%	100%
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty Total Marks Awarded:

ANALYTICAL PROBLEM SOLVING — ANSWERS

1. A cloud datacenter host has 64 CPU cores and supports CPU overcommitment at a ratio of 2.5:1. Each virtual machine (VM) requires 4 vCPUs. Calculate the maximum number of VMs that can be deployed on this host. If actual CPU utilization reaches 85%, analyze whether further VM allocation is advisable.
Solution:
Total allowable vCPUs = Physical cores × Overcommitment ratio = 64 × 2.5 = 160 vCPUs
Maximum number of VMs = Total allowable vCPUs ÷ vCPUs per VM = 160 ÷ 4 = 40 VMs
Analysis at 85% utilization: An actual CPU utilization of 85% is already close to saturation. Allocating further VMs at this point risks CPU contention, increased scheduling latency, and degraded performance for existing workloads, even though the overcommitment ratio mathematically still permits more VMs. Therefore, further VM allocation is not advisable until utilization is reduced (through load balancing, migrating VMs to another host, or scaling out) — the host should instead be monitored and capacity expanded before onboarding new VMs.

2. A virtualized storage system provides 20 TB of physical storage with thin provisioning at a ratio of 3:1. Each VM is allocated 500 GB of virtual disk space. Calculate the maximum number of VMs that can be provisioned.
Solution:
Total provisionable (virtual) storage = Physical storage × Thin provisioning ratio = 20 TB × 3 = 60 TB = 60,000 GB
Maximum number of VMs = Total provisionable storage ÷ Storage per VM = 60,000 GB ÷ 500 GB = 120 VMs
Note: Since thin provisioning allocates storage on demand rather than upfront, this figure represents the theoretical maximum; actual safe capacity should be monitored to avoid over-allocation beyond the 20 TB physical limit if VMs consume their full allocated space simultaneously.

3. Evaluate the security implications of multi-tenancy in cloud datacenters. Analyze how virtualization can both enable and weaken isolation, and recommend mechanisms to strengthen tenant separation.
Multi-tenancy allows multiple customers (tenants) to share the same underlying physical infrastructure, which introduces both benefits and risks for security.
How virtualization enables isolation: The hypervisor abstracts physical hardware into separate virtual machines, each with its own operating system, memory space, virtual network, and storage. This logical separation prevents tenants from directly accessing each other's data or processes under normal operation.
How virtualization weakens isolation: Because tenants share the same physical CPU, memory, storage, and hypervisor layer, vulnerabilities can still be exploited, for example through hypervisor escape attacks (a VM breaking out to affect the host or other VMs), side-channel attacks (inferring data through shared caches or resource usage patterns), and "noisy neighbor" issues where one tenant's resource consumption degrades performance for others.
Recommended mechanisms to strengthen tenant separation:

- Use hardware-assisted virtualization security features (e.g., Intel VT-x/AMD-V) to enforce stronger boundaries between VMs.
- Apply network micro-segmentation and VLAN/VXLAN isolation so tenant traffic cannot cross boundaries.
- Enforce strict identity and access management (IAM) with role-based access control per tenant.
- Keep the hypervisor patched and hardened, and use resource quotas/limits to prevent noisy-neighbor effects.
- Encrypt data at rest and in transit, and use continuous monitoring or intrusion detection to catch cross-tenant anomalies early.

4. Analyze the role of Service-Oriented Architecture (SOA) in enabling interoperability across heterogeneous cloud platforms. Identify key challenges such as service latency, security, and version control, and propose solutions.
Role of SOA: SOA structures functionality as independent, loosely coupled, reusable services that communicate through standardized interfaces and protocols (such as SOAP or REST, using XML/JSON). Because each service exposes a well-defined contract rather than depending on a specific underlying platform or language, SOA allows heterogeneous cloud systems — built on different vendors, operating systems, or technology stacks — to interact and exchange data seamlessly, which is the foundation of interoperability in cloud computing.
Key challenges and proposed solutions:

- Service latency: Chaining multiple services together over a network introduces delay. This can be reduced using caching, API gateways, asynchronous messaging, and optimizing service granularity to minimize unnecessary calls.
- Security: Exposing many service endpoints across organizational boundaries increases the attack surface. This can be addressed with token-based authentication (e.g., OAuth), API gateways enforcing consistent security policies, and end-to-end encryption.
- Version control: As services evolve, changes can break dependent consumers. This is managed through clear service contracts (e.g., WSDL/OpenAPI specifications), maintaining backward-compatible versions, and using a service registry to track and govern versions during rollout.

5. A cloud system implementing presence services faces scalability issues during peak usage. Analyze the architectural bottlenecks and recommend design improvements to ensure real-time updates and efficient resource utilization.

Architectural bottlenecks: Presence services typically suffer at peak load due to a centralized presence server acting as a single point of failure and contention, frequent small writes causing database contention, reliance on inefficient polling by clients to check status (creating high request overhead), and a lack of horizontal scalability in the architecture to absorb sudden spikes in concurrent users.

Recommended design improvements:

- Replace polling with a publish-subscribe model using persistent connections (e.g., WebSockets) or lightweight messaging protocols (e.g., MQTT) so updates are pushed only when status actually changes.
- Use an in-memory data store (e.g., Redis) for presence state to enable fast reads/writes instead of relying on a traditional disk-based database.
- Horizontally scale presence servers behind a load balancer, and shard/partition users across nodes to distribute load evenly.
- Introduce a distributed message broker (e.g., Kafka) to decouple presence update producers from consumers and absorb traffic spikes.
- Use geographically distributed edge servers to reduce latency and balance load closer to users during peak regional usage.